

Dev / Engineering (MVP Skeleton)

1. Tech stack

- Frontend: Next.js (App Router) + TypeScript
 - Styling: Tailwind CSS
 - Backend: Supabase (Postgres + Auth + Row-level security)
 - Deployment: Vercel (or similar)
 - Platform: Web-first, mobile-optimized (PWA-capable later)
-

2. High-level features (MVP)

- User auth (email/password or magic link)
 - Today Check-In form (waking + post-meal temp/pulse + notes)
 - Daily entries list (last 14–30 days)
 - GQ calculation (14/30/90 day) and tier display
 - Pattern label (Cold & Slow / Hot but Wired / Warm & Steady)
 - Simple charts for the four metrics
 - Basic reminder settings (time, days) – even if only stored, not yet fully scheduled
-

3. Routes / pages (Next.js)

/ – Landing

- Hero: explanation of Ascentgen, GQ, and patterns.
- CTA: “Enter Ascentgen” (to /app or /login).
- Very minimal, instrument-like.

[/login](#) / [/signup](#)

- Auth forms using Supabase Auth.
- Magic link or email/password.

[/app](#) – **Main dashboard (protected)**

Sections:

1. Header

- Brand mark, GQ tier label, date.

2. GQ Card

- Current 14-day GQ score + tier (Dormant / Kindling / Ascending).
- Pattern label (Cold & Slow / Hot but Wired / Warm & Steady).
- One-line explanation.

3. Today Check-In

- Form:
 - Waking temp (value + unit).
 - Waking pulse.
 - Post-meal temp (value + unit).
 - Post-meal pulse.
 - Notes (optional).
- Button: "Lock in today's readings".

4. Recent Entries

- Table/list for last 14 days:
 - Date, waking temp/pulse, post-meal temp/pulse.
- Optional: inline edit.

5. Trends

- Small charts (14–30 days) for each metric.
- GQ history mini-chart.

`/settings` – Settings (protected)

- Units (°C / °F).
 - Time zone.
 - Reminder times:
 - Morning baseline window.
 - Post-meal reminder delay (45–60 min).
-

4. Database schema (Supabase)

Table: `profiles`

- `id` (uuid, PK, references auth.users.id)
- `email` (text)
- `created_at` (timestamp)
- `timezone` (text)
- `temperature_unit` (text, 'C' | 'F', default 'F')
- `morning_reminder_time` (time, nullable)
- `post_meal_reminder_offset_minutes` (int, default 60)

Table: `daily_entries`

- `id` (uuid, PK)
- `user_id` (uuid, FK → profiles.id)
- `date` (date)
- `waking_temp_value` (float, nullable)
- `waking_temp_unit` (text, 'C' | 'F' | null)
- `waking_pulse_bpm` (int, nullable)
- `post_meal_temp_value` (float, nullable)
- `post_meal_temp_unit` (text, 'C' | 'F' | null)
- `post_meal_pulse_bpm` (int, nullable)

- `notes` (text, nullable)
- `created_at` (timestamp, default now())
- `updated_at` (timestamp, default now())

Indexes

- Unique per user per date: (`user_id`, `date`).
- Index on `user_id` for fast queries.

Table: `gq_snapshots` (optional v1, nice for caching)

- `id` (uuid, PK)
- `user_id` (uuid, FK)
- `window_days` (int – 14, 30, 90)
- `start_date` (date)
- `end_date` (date)
- `gq_score` (int)
- `gq_tier` (text: 'Dormant' | 'Kindling' | 'Ascending')
- `pattern_label` (text: 'Cold & Slow' | 'Hot but Wired' | 'Warm & Steady')
- `created_at` (timestamp)

For v1, you can compute GQ on the fly and skip this table; add it later if performance becomes an issue.

5. GQ calculation (implementation notes)

- Logic lives in `app/lib/gq.ts`.
- Uses 14/30/90-day windows of `daily_entries` for:
 - Waking/post-meal temp and pulse (converted to °C internally).
 - Closeness and stability scores (v0.2 thresholds from Bible).
- Returns:
 - `gqScore`, `gqTier`, `patternLabel`, and per-metric stats.

When refining, always update:

1. **Ascentgen Bible – GQ implementation section.**
2. **Code in `gq.ts`.**

Then re-sync with Claude and here.

6. Frontend component map

Inside `/app` route:

- `components/GQCard.tsx`
 - Displays GQ score, tier, pattern, and a one-liner.
 - `components/TodayCheckInForm.tsx`
 - Fields for all four readings + notes.
 - Handles submission to `/api/log-today`.
 - `components/RecentEntriesList.tsx`
 - Table/list with last 14 days.
 - Optional inline editing.
 - `components/MetricChart.tsx`
 - Generic chart component for temp/pulse (pass metric + data).
 - `components/Layout/AppShell.tsx`
 - Header, main column layout, basic navigation.
-

7. API routes (Next.js)

POST `/api/log-today`

- Auth required.
 - Upsert by (`user_id`, `date`).
 - Body: waking/post-meal temps + units + pulses + notes.
 - Returns updated `daily_entries` row.
-

`GET /api/daily-entries`

- Query params: `windowDays` (e.g., 14, 30).
- Returns recent entries ordered by `date DESC`.

`GET /api/gq`

- Query params: `windowDays`.
- Server computes GQ from `daily_entries` using `gq.ts`.
- Returns `score`, `tier`, `pattern`, and optionally the underlying stats.

(You can also compute GQ directly in server components instead of a dedicated API route; this is just a clean separation.)

8. Integration with AI workflow

Add this small subsection to remind yourself how to use the two AIs against this dev spec:

- When designing or changing routes, components, or DB schema:
 - Take this `Dev / Engineering` page + Ascentgen Bible to **Claude 3.5 Sonnet**.
- Ask Claude to:
 - Propose file organization and code.
 - Generate TS interfaces, Supabase migration SQL, and component skeletons.
- When Claude proposes anything that touches:
 - Measurement protocol.
 - GQ math.
 - Any health phrasing.
 - Bring it here for review and final wording with citations.
- After review, update:
 - This page (Dev spec).
 - Ascentgen Bible (if conceptual change).